

Consulting Form & AI Enhancement Guide

This document provides a beginner-friendly, step-by-step setup for the Portfolio Site Consulting Intake Form. This project involves fixing a broken webhook and integrating Google Gemini AI to send personalized emails.

Project Overview

- **Project:** Portfolio Site — Consulting Intake Form
- **Author:** Derrick Odiwuor
- **Date:** August 2026
- **Primary Goal:** Ensure every "Book Setup Sprint" form submission triggers a personalized, AI-generated email to both the client and the admin.

Step 1: Environment Configuration

Before the code can send emails or talk to external services, you must define your "secrets" in an environment file.

Local Setup

Create or update a .env.local file in your root directory with the following variables:

Variable Name	Purpose
MAKE_COM_WEBHOOK_URL	Connects the form to Make.com for automation.

Variable Name	Purpose
GEMINI_API_KEY	Powers the AI-generated personalized messages.
SMTP_PASSWORD	The App Password for your Gmail account (Nodemailer).
NEXT_PUBLIC_APP_URL	The URL of your website (e.g., <code>http://localhost:3000</code>).

Production Setup

When deploying to **Vercel**, navigate to **Settings > Environment Variables** and add these same keys to ensure the live site functions correctly.

Step 2: Fixing the Webhook

If form submissions are not appearing in your automation tool (Make.com), the webhook is likely missing or disconnected.

1. **Locate the Route:** Open `app/api/consulting/submit/route.ts`.
2. **Verify the Variable:** Ensure the code is looking for `PROCESS.ENV.MAKE_COM_WEBHOOK_URL`.
3. **The Fix:** If the variable was missing from your `.env` file, the system would silently skip the alert. Adding the URL to your environment variables solves this immediately.

Step 3: Implementing AI Personalization

We use **Google Gemini 1.5 Flash** to read form entries and write unique responses.

How it Works

1. **Data Collection:** The system takes the user's name, company, and project goals.
2. **AI Prompting:** It sends this data to Gemini with instructions like: *"Write a professional email to [Name] at [Company] discussing their interest in [Project Goals]."*
3. **Dual Delivery:**
 - **Client Email:** A warm, personalized confirmation sent to the applicant.
 - **Admin Email:** A summary sent to  **Person** .

Fallback Protection

If the AI service is down or the API key expires, the system is designed to switch to a **Professional Template**. This ensures the user always receives a confirmation, even if the AI isn't available.

Step 4: Formatting & Refinements

Based on recent updates, the following visual and structural changes have been applied:

- **Human-Readable Admin Alerts:** Admin notifications no longer show messy code (JSON). They are now formatted as clean, labeled text.
- **Simplified Design:** Removed all "Onboarding Portal" links to keep the focus on the initial consultation.
- **Professional Subject Lines:** Removed emojis (like ) from email subject lines to maintain a corporate aesthetic.

Step 5: Testing the Setup

Follow this checklist to ensure everything is working perfectly.

Local Testing Checklist

- ☐ Run `npm run dev` and ensure no terminal errors appear.

- ☐ Submit a test entry through the "Book Setup Sprint" form.
- ☐ Confirm the browser redirects to your Calendly page.
- ☐ Check your inbox for the **Admin Notification**.
- ☐ Check the test email you used for the **Client Confirmation**.
- ☐ Verify the emails sound "human" (AI-generated) rather than generic.

Production (Vercel) Checklist

- ☐ Confirm all environment variables are added to the Vercel Dashboard.
- ☐ Deploy the latest changes.
- ☐ Perform one "Live" submission to verify the Make.com webhook receives data.

Maintenance & Updates

- **To Change AI Tone:** Edit the prompt strings in `lib/email.ts`.
- **To Change Email Style:** Update the HTML templates within the `generate` functions in `lib/email.ts`.
- **To Add Form Fields:** Update the UI in `intake-form.tsx`, add the field to the Zod schema in `validation.ts`, and pass the new data to the AI prompt in the submit route.